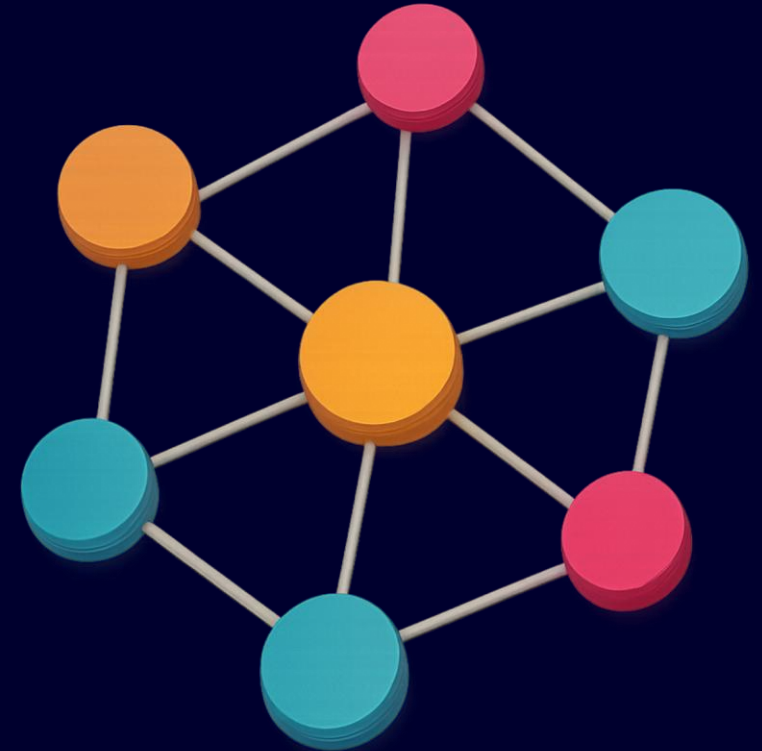


Desarrollo de Software en Sistemas Distribuidos

Licenciatura en Sistemas · Universidad Nacional de Lanús · 2026

Aguardamos unos minutos a que ingresen a la sala



Componentes Típicos de una Aplicación

Toda aplicación de software se estructura en tres capas funcionales bien diferenciadas. Comprender cada una es el punto de partida para analizar cualquier arquitectura.



Interfaz de Usuario

Elemento con el que el usuario interactúa: ejecuta acciones, introduce datos y obtiene información del sistema.



Lógica de Negocio

Procesa la información para generar los resultados esperados. Es el núcleo que diferencia una aplicación de otra.



Gestión de Datos

Se ocupa del almacenamiento persistente y la recuperación eficiente de la información del sistema.



ARQUITECTURA MONOLÍTICA

Aplicaciones Monolíticas

En una aplicación monolítica, las tres capas –interfaz, lógica de negocio y gestión de datos– forman un único bloque indivisible que se ejecuta en un **mismo paquete o proceso**.

- ❏ El usuario y el sistema coexisten en un mismo entorno: no hay separación entre cliente y servidor.

Todo en Cada Máquina

Cuando la aplicación monolítica se instala en múltiples equipos, **todos los componentes se replican** en cada estación de trabajo: interfaz, lógica y acceso a datos.

Redundancia total

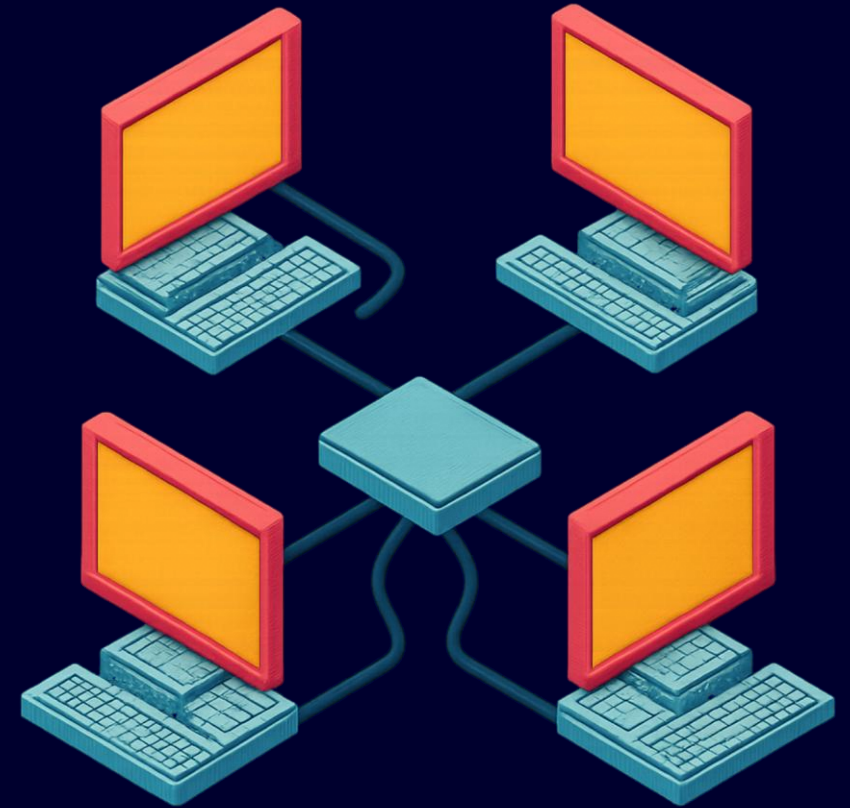
Cada máquina almacena y procesa localmente, generando tráfico innecesario en la red.

Acceso directo a datos

Los usuarios necesitan permisos reales sobre la carpeta compartida donde residen los datos.

Escalabilidad limitada

El crecimiento del sistema implica actualizar manualmente cada estación de trabajo.



Ventajas y Desventajas del Modelo Monolítico

Ventajas

- Funcionan rápido en entornos locales controlados
- Fáciles de desarrollar: todo está en un solo proyecto
- Menor complejidad inicial de infraestructura

Desventajas

- Requieren hardware potente en cada estación
- Ocupan mayor ancho de banda, congestionando la red
- Su actualización es costosa y disruptiva

¿Por Qué Construir Sistemas Distribuidos?

Los sistemas distribuidos surgen como respuesta a las limitaciones del modelo monolítico, habilitando nuevas posibilidades de diseño y operación.

Compartir recursos

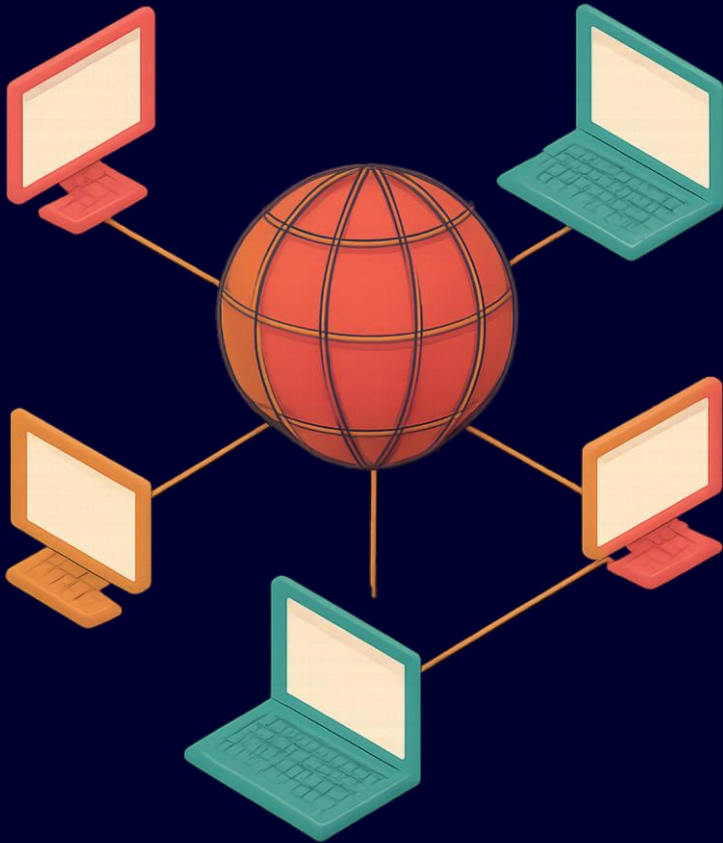
Hardware, almacenamiento y servicios pueden ser utilizados por múltiples nodos de forma coordinada.

Diseño modular

Cada componente se ejecuta en la plataforma más adecuada para su función específica.

Servicio eficiente

La especialización de cada parte permite optimizar rendimiento, seguridad y disponibilidad.



Definiciones Clásicas de Sistemas Distribuidos

Distintos autores han abordado el concepto desde perspectivas complementarias, pero convergentes.

Coulouris (2000)

Un sistema distribuido es un conjunto de computadoras independientes que aparecen ante los usuarios como un único sistema coherente.

Tanenbaum (2007)

Una colección de computadoras autónomas interconectadas por una red, que cooperan mediante el intercambio de mensajes.

Otros autores

Enfatizan la transparencia: el usuario no debe percibir la distribución subyacente del sistema.

Otra definición posible...

Un sistema distribuido es una colección de computadoras autónomas conectadas a través de una red, con el software adecuado para que el sistema sea visto por los usuarios como una única entidad capaz de proporcionar facilidades de cómputo.

Autonomía

Cada nodo opera de forma independiente

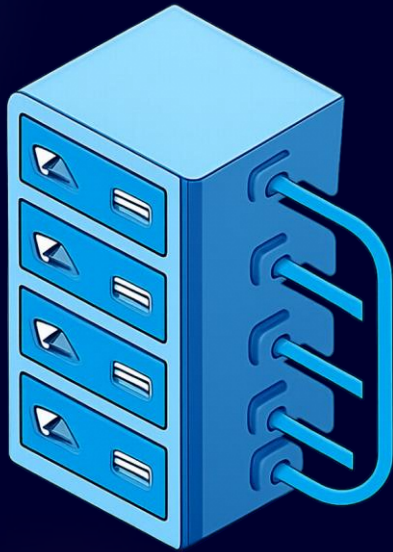
Red

Comunicación mediante mensajes

Transparencia

El usuario percibe un sistema único

Características Fundamentales



→ *Concurrencia*

Múltiples procesos se ejecutan simultáneamente en distintos nodos.

→ *Ausencia de reloj global*

No existe un estado temporal compartido entre todos los componentes.

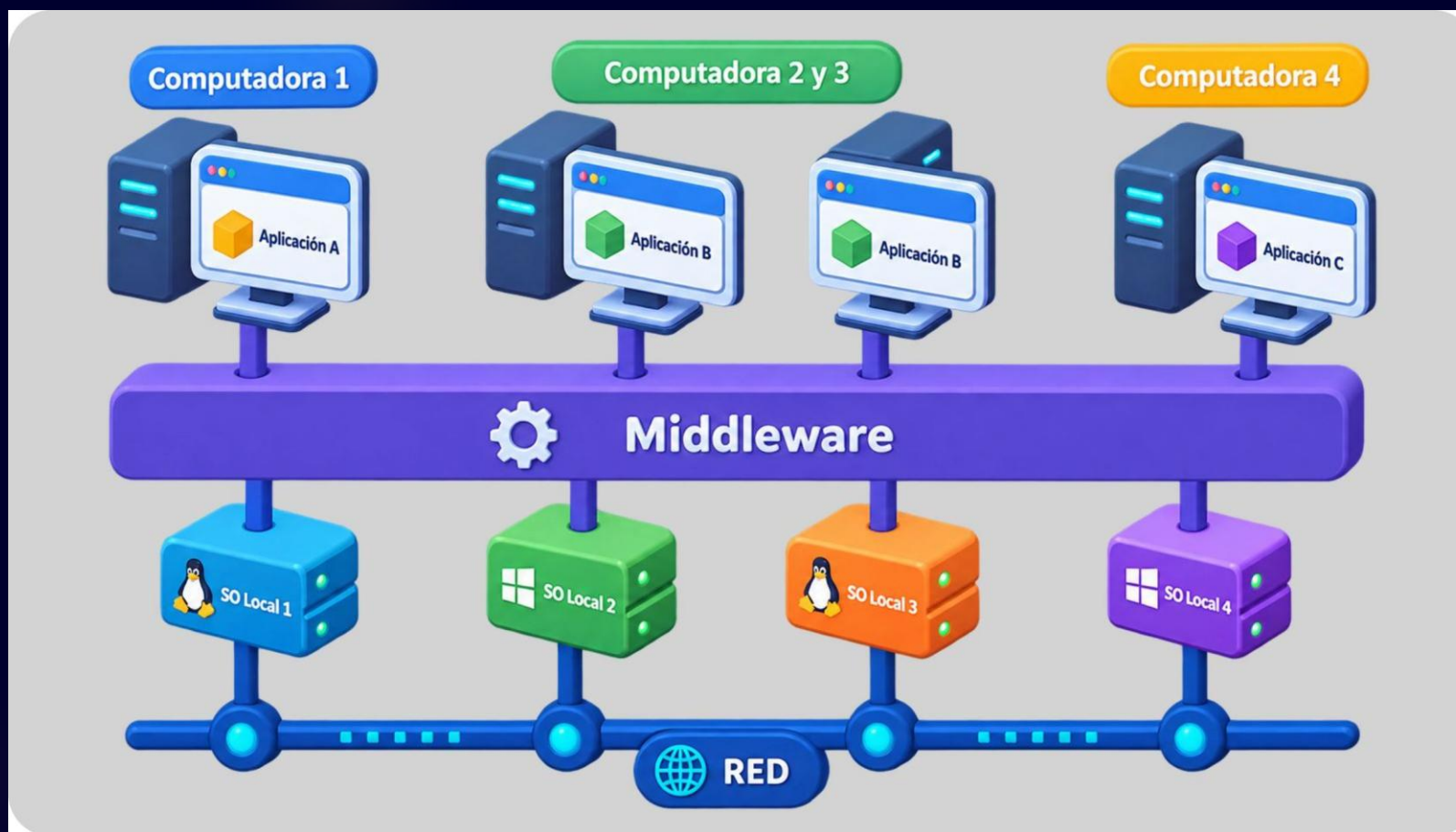
→ *Fallos independientes*

Un nodo puede fallar sin que el sistema completo colapse.

→ *Transparencia*

El usuario no percibe la complejidad de la distribución subyacente.

No necesariamente debe existir una correspondencia unívoca entre computadoras y aplicaciones. También se observa que la visión del sistema único es lograda por la capa middleware que oculta los detalles de cada computadora.



Ejemplos de Sistemas Distribuidos

Los sistemas distribuidos están presentes en la vida cotidiana y en la infraestructura crítica del mundo digital.



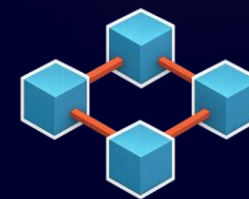
Internet

El mayor sistema distribuido: millones de nodos cooperando para entregar información globalmente.



Computación en la Nube

Plataformas como AWS, Azure o Google Cloud distribuyen carga, almacenamiento y servicios.



Blockchain

Redes descentralizadas donde cada nodo valida y almacena una copia del registro compartido.

Seis Características Clave



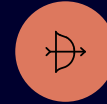
Compartición de Recursos

Hardware y software compartidos



Apertura

Sistema extensible



Concurrencia

Procesos en paralelo



Escalabilidad

Funciona a distintas escalas



Tolerancia a Fallos

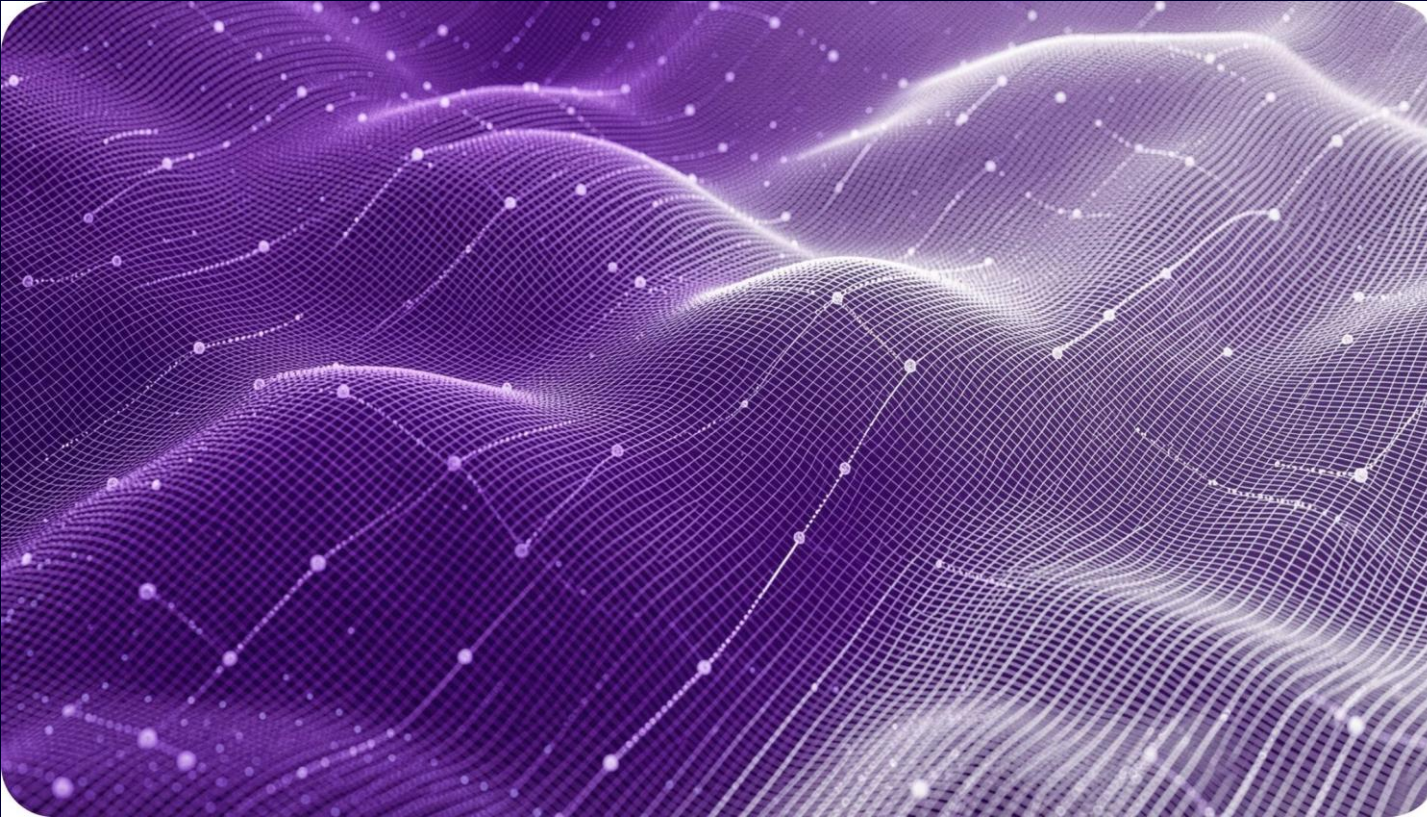
Sigue operando ante fallos



Transparencia

La distribución se oculta

Compartición de Recursos



¿Qué es un recurso?

Un recurso es una entidad compartible: hardware, archivos, bases de datos u objetos de datos.

Gestor de Recursos

Administra recursos del mismo tipo y controla su acceso y actualización.

Modelos Abstractos

Dos modelos principales: cliente-servidor y basado en objetos.



Apertura (Openness)

Extensibilidad

Un sistema abierto puede ampliarse en hardware y software.

Interfaces Públicas

Las interfaces están bien especificadas para que los desarrolladores añadan servicios.

Heterogeneidad

Distintos vendedores funcionan juntos si siguen los estándares publicados.

Concurrencia

Ejecución Paralela

En sistemas distribuidos con **M** ordenadores, hasta **M** procesos pueden ejecutarse en paralelo. Esa es una de las grandes ventajas frente a un sistema centralizado.

Sincronización en Servidores

Cuando varios procesos servidor atienden peticiones a la vez, deben sincronizarse para evitar conflictos. La coordinación correcta mantiene la concurrencia sin perder control.



Escalabilidad

Diseño para el Crecimiento

Los sistemas distribuidos deben mantenerse eficientes y escalables desde unas pocas máquinas hasta miles de computadoras conectadas globalmente.

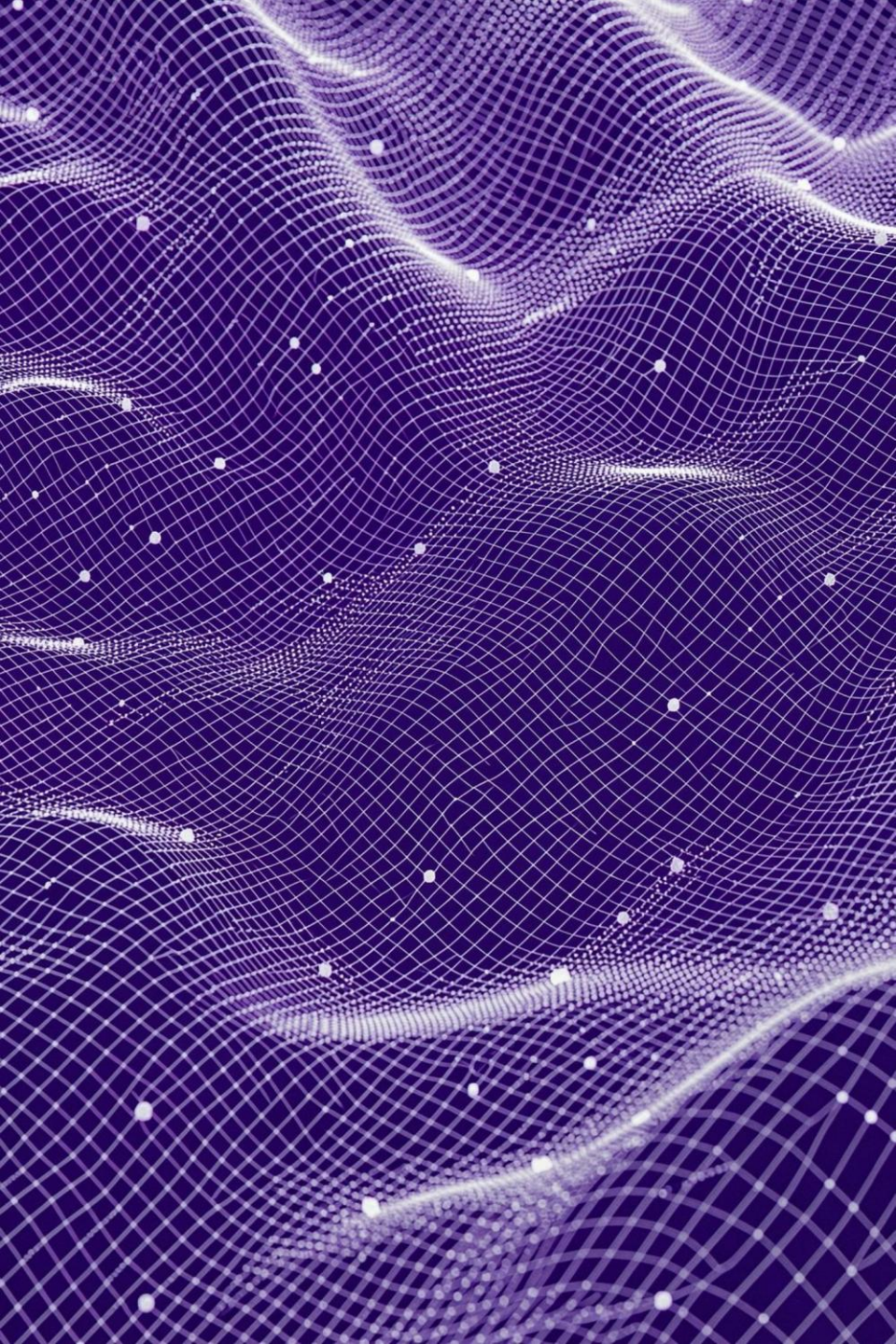
Técnicas Clave

- **Replicación de datos:** Múltiples copias para alta disponibilidad
- **Caching:** Almacenamiento temporal para reducir latencia
- **Múltiples servidores y concurrencia:** Distribución de carga y mayor productividad



Escalabilidad sin límites (o casi)

Desde 2 nodos hasta millones – el diseño escalable es lo que diferencia los sistemas distribuidos de los sistemas centralizados.



Tolerancia a Fallos

Redundancia Hardware

Los componentes redundantes mantienen el sistema en funcionamiento aunque falle una pieza.

Recuperación Software

Los programas se recuperan de fallos mediante **rollback** al último estado estable o mediante reintentos.

Alta Disponibilidad

Un fallo solo afecta a ese componente; los usuarios pueden cambiar a otra estación o servidor.

Transparencia

La transparencia consiste en ocultar la separación entre componentes para que el sistema se perciba como un todo unificado.



Acceso y Localización

Acceder a objetos remotos como si fueran locales, sin conocer su ubicación.



Fallos y Migración

Completar tareas pese a fallos y mover objetos sin afectar a usuarios ni aplicaciones.



Concurrencia y Replicación

Múltiples procesos e instancias operan sin interferencia y de forma transparente.



Prestaciones y Escalado

Reconfigurar para mejorar prestaciones y expandir el sistema sin cambiar su estructura.



Arquitectura Mainframe (Centralizada)

Definición

Sistema centralizado donde todos los recursos y la lógica de aplicación residen en una única computadora central. Modelo multiusuario con control y acceso compartido centralizados.

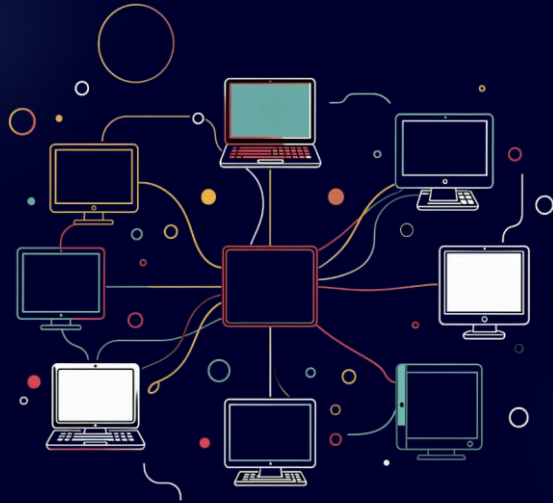
Limitaciones

Punto único de fallo, escalabilidad limitada y dependencia total de una máquina central hacen de este modelo un cuello de botella para sistemas en crecimiento.

Contexto Histórico

Modelo dominante antes del desarrollo de redes locales de alta velocidad en los años 70. Su declive abrió la puerta a los sistemas distribuidos.

Arquitectura Cliente-Servidor: La Alternativa



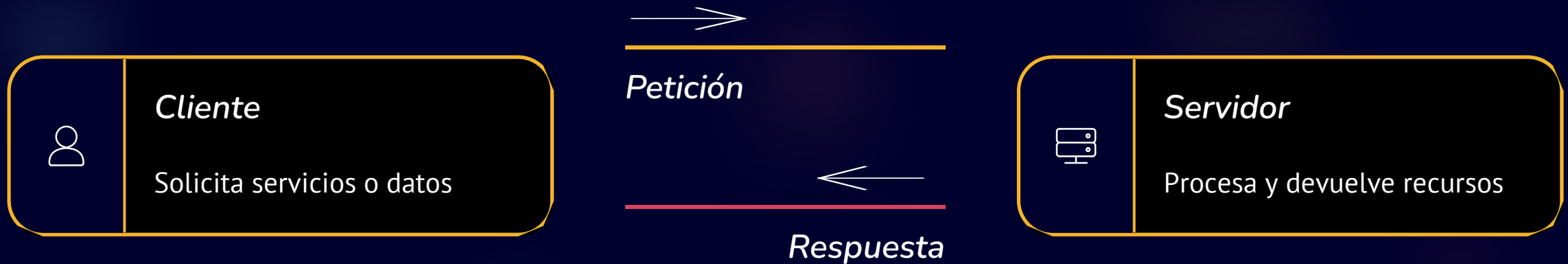
Definición

El modelo cliente-servidor es el más conocido y más ampliamente adoptado en la actualidad. Existe un conjunto de **procesos servidores**, cada uno actuando como gestor de recursos, y una colección de **procesos clientes**, cada uno llevando a cabo tareas que requieren acceso a recursos hardware y software compartidos.

Ventajas Clave

- Distribución de responsabilidades y recursos a través de la red
- Procesos cliente y servidor pueden residir en máquinas diferentes o en la misma máquina
- Flexibilidad en la asignación de recursos

Flujo de Comunicación Cliente-Servidor



Los **gestores de recursos** son módulos software que manejan conjuntos de recursos de un tipo particular.

El flujo de comunicación es bidireccional: el cliente inicia la solicitud y el servidor responde con los datos o servicios requeridos, manteniendo una separación clara de responsabilidades.

Servidores Pesados vs. Clientes Pesados

Fat Server

- Mayor encapsulación de datos
- Compatibilidad mejorada entre clientes
- Clientes más simples y ligeros
- Ejemplo clásico: servidor Web

Fat Client

- Mayor responsabilidad en el cliente
- Más fáciles de implementar
- Permiten crear y modificar aplicaciones fácilmente
- Requieren conocimiento íntimo de los datos



Distribución de la Lógica de Aplicación

Ventajas

- Flexibilidad en el diseño del sistema
- Actualización independiente de componentes
- Mejor aprovechamiento de recursos
- Escalabilidad mejorada

Desventajas

- Complejidad aumentada en la comunicación
- Sincronización entre componentes
- Mayor latencia de red
- Dificultad en mantener consistencia de datos



Dónde colocar la lógica — en el cliente, en el servidor, o compartida entre ambos — define la naturaleza del sistema distribuido.

Sistemas N-Tiered: Two-Tier y Three-Tier



Two-Tier

Cliente y servidor como únicos participantes. La lógica de la aplicación puede estar en el cliente, el servidor, o compartida entre los dos.



Three-Tier

Presentación, lógica de negocio y datos completamente separados en capas independientes. Mayor modularidad y mantenibilidad.



N-Tier

Extensión del modelo three-tier con capas adicionales para cubrir necesidades de sistemas complejos a escala empresarial.

En un sistema **three-tiered** ideal, toda la lógica de la aplicación reside en una capa separada, desacoplada tanto de la interfaz de usuario como del almacén de datos.

Arquitectura Web: Ejemplo Clásico de 3 Capas



Capa de Presentación

La interfaz de usuario es provista por la interpretación de HTML a través de un navegador. Es la capa visible para el usuario final.

Capa de Lógica de Aplicación

Los componentes embebidos visualizados por el navegador residen en la capa media, proveyendo toda la lógica de aplicación del sistema.

Capa de Datos

Los datos son suministrados por el servidor Web. Esta separación garantiza mantenimiento independiente de cada capa.



Ventajas de los Sistemas Cliente-Servidor



Flexibilidad Natural

Las aplicaciones encajan de manera natural en el marco cliente-servidor. Por ejemplo, una agenda electrónica con datos relativamente estáticos se beneficia de colocar dichos datos en un servidor que acepta peticiones de todos los usuarios de manera uniforme.



Interoperabilidad

Gracias a la disponibilidad de middleware compatible para múltiples plataformas, los sistemas cliente-servidor pueden conectar clientes ejecutándose en una plataforma con servidores en una plataforma completamente distinta.



Escalabilidad

Es posible ejecutar aplicaciones de uso intensivo de recursos en hardware de bajo coste. El sistema es más escalable: se pueden añadir tanto nuevos clientes como nuevos servidores según la demanda.

Conclusión: Encapsulación y Diseño Orientado a Interfaces

Si las porciones de un sistema cliente-servidor **encapsulan una única función** y siguen una **interfaz perfectamente definida**, aquellas partes que proveen los servicios pueden ser intercambiadas sin afectar a otras porciones del sistema.

Beneficios de la Encapsulación

- Independencia entre componentes
- Facilidad para realizar cambios y actualizaciones
- Adaptabilidad a nuevas necesidades
- Mantenimiento simplificado



El diseño orientado a interfaces permite que usuarios, desarrolladores y administradores adapten el sistema según sus necesidades en cada momento, resultando en sistemas distribuidos **robustos, flexibles y escalables**.

Fuentes

- PDF: Introducción a los Sistemas Distribuidos